

Problem 1

Instructions for the Code (Problem 1 Version1.ipynb)

The following steps are the steps of implementing the code:

1. First we import all the needed packages such as gym, gym, numpy, matplotlib.pyplot, tensorflow, keras, tensorflow_probability, warnings, & math.
2. Next we define the environment using the code below such that it allows us to simulate the game CartPole, version 0. This environment is a built-simulated environment using gym.

```
env = gym.make("CartPole-v0")
```

3. Next, we build a simple neural network. Since the game doesn't include many variables; the model below is sufficient. The model includes two fully connected layers with 32 neurons and a linear activation function, then 12 neurons and action function relu, respectively. The output layer has an activation function: SoftMax.

Linear activation function: It has the formula $f(z) = z$, for each input vector z .

ReLU activation function: replaces all negative values in the input with zero and leaves positive values unchanged. For each input vector z , RELU gives $f(z) = \max(0, x)$.

Softmax activation function: is a mathematical function that is often used to convert a vector of raw scores or logits into a probability distribution, where the goal is to assign an input to one of several possible classes. For each element i in the input vector z :

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

```
model = keras.Sequential([
    keras.layers.Dense(32, activation='linear', input_shape=(4,)),
    keras.layers.Dense(12, activation='relu', input_shape=(32,)),
    keras.layers.Dense(2, activation='softmax')]) # Two actions: Left and right])
```

This neural network is used to train the trainable values (θ , of the policy that is used to play the game, meaning decide between the two actions of the game, left or right.

4. Next an optimizer is defined using keras. Adam is selected with a learning rate = 0.005.

```
# optimizer for training
optimizer = keras.optimizers.Adam(learning_rate=0.005)

# discount factor
gamma = 0.95
```

5. The discount factor $\gamma = 0.95$ is defined.
6. Lists are initiated outside of the training loop to:
 - track the rewards of the last 100 episodes

- Track all the rewards of all the episodes

```
last_100_episodes = []  
episode_rewards = []
```

7. A for loop is started that runs over batches of episodes. At each batch, a certain number of episodes is sampled, in this case (20). Next, empty lists are initiated to keep track of actions, rewards, observations (these weren't used afterwards in the code). And at the beginning of the batch, the loss is initiated to 0. Afterwards `tf.gradienttape` is used to keep track of the functions and the model, to be able to compute the gradient afterwards:

The `tf.GradientTape()` is a context manager provided by TensorFlow that is used for automatic differentiation, specifically for computing gradients. Context Management: When you enter a block of code using with `tf.GradientTape()` as `tape`:, you are creating a context where TensorFlow will "record" the operations that occur inside this block. It's like turning on a tape recorder for TensorFlow to keep track of what's happening. TensorFlow uses this recorded information to automatically compute gradients of operations with respect to their inputs.

With each loop across the batch, another loop is created to loop over episodes.

```
for batch in range(0, 200):  
    num_episodes = 20  
    # Training loop  
    obs_history_batch = []  
    reward_history_batch = []  
    action_history_batch = []  
    discounted_rewards_batch = []  
    loss = 0  
    with tf.GradientTape() as tape:  
        #tape.watch(model.trainable_variables)  
  
        for episode in range(num_episodes):
```

- 7.1. Within each loop of the episode, we initiate the following lists and variables:

```
obs_history_eps = []  
reward_history_eps = []  
action_history_eps = []  
selected_probs_eps = []  
  
terminated = False  
truncated = False
```

Afterwards, the loop where the episode takes place is initiated.

- ```
while (not terminated) and (not truncated):
 # sampling an action from the policy
 obs_array = np.array(obs_history_eps[-1] if obs_history_eps else env.reset()[0],
 obs_array.reshape((1, 4)) # Reshape to 1D array
 obs_tensor = tf.convert_to_tensor(obs_array)
 prob = model(obs_tensor)
```

- ```
dist = tfp.distributions.Categorical(probs=prob, dtype=tf.float32)
#print("distribution", dist)
action = dist.sample()
#print("selected proba", prob[0][int(action)])

obs, reward, terminated, truncated, info= env.step(int(action.numpy()[0]))

obs_history_eps.append(obs)
reward_history_eps.append(reward)
action_history_eps.append(action)
selected_probs_eps.append(prob[0][int(action)])

obs_history_batch.append(obs)
reward_history_batch.append(reward)
action_history_batch.append(action)
```

- 7.2. Outside of the while loop, once the episode ends, the sum of the rewards of the episode is tracked.

- 7.3. Then the discounted rewards of that episode is computed in a vector such that, we loop in the reverse order throughout the rewards of the episodes, and compute the discounted reward of the last state. Then for the state before the last, we add the reward sum of the last and add the discounted reward of the corresponding state, and so on. This vector will include the discounted reward for each state, where it include the rewards to go from that state.

```
discounted_rewards_eps = []
reward_sum = 0
for r in reward_history_eps[::-1]:
    reward_sum = r + gamma * reward_sum
    discounted_rewards_eps.insert(0, reward_sum)
#print(discounted_rewards)

discounted_rewards_eps = np.array(discounted_rewards_eps)
```

- 7.4. The discounted rewards vector is converted to a NumPy array with shape (T, 1), where T is the number of states in an episode.
- 7.5. Next, we convert the discounted rewards to a tensor, and we convert the vector of selected probabilities into a tensor.
- 7.6. We get the log of the probabilities.
- 7.7. We add the loss computed for this episode to the one computed for previous episodes in the same batch.

```
reshaped_discounted_rewards = discounted_rewards_eps.reshape(-1, 1)
#epsilon = 1e-8
#log_probs = tf.math.log(selected_action_probs + epsilon)
#print(log_probs)
tensor_logprob = tf.math.log(tf.convert_to_tensor(selected_probs_eps, dtype=tf.float32))
tensor_rewards = tf.convert_to_tensor(reshaped_discounted_rewards, dtype=tf.float32)

loss = loss + tf.reduce_sum (tensor_logprob * tensor_rewards)
#print(loss)
```

- 7.8. For each batch, we convert the loss of the batch to negative, as the default process is to minimize the loss, and in this case, we want to maximize the loss which represents the reward).
- 7.9. We get the gradient of the of the policy

```
#LOSS = -LOSS
loss = -tf.convert_to_tensor(loss)
grads = tape.gradient(loss, model.trainable_variables)
```

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t) R[t] \right]$$

7.10. Then we apply the gradient using the optimizer:

```
optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

8. We create plots, print values of rewards every 100 episodes, and close the environment.

```
if len(episode_rewards) >= 100:
    avg_reward = np.mean(episode_rewards[-100:])
    last_100_episodes.append(avg_reward)
    print(f"Average Reward (Last 100 Episodes): {avg_reward}")

# Plot the rewards

plt.plot(episode_rewards)
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.title('Rewards vs. Episodes')
plt.show()
```

[Updates to Problem1_Updated.ipynb](#)

A few updates have been made between Version 1 and the Updated code:

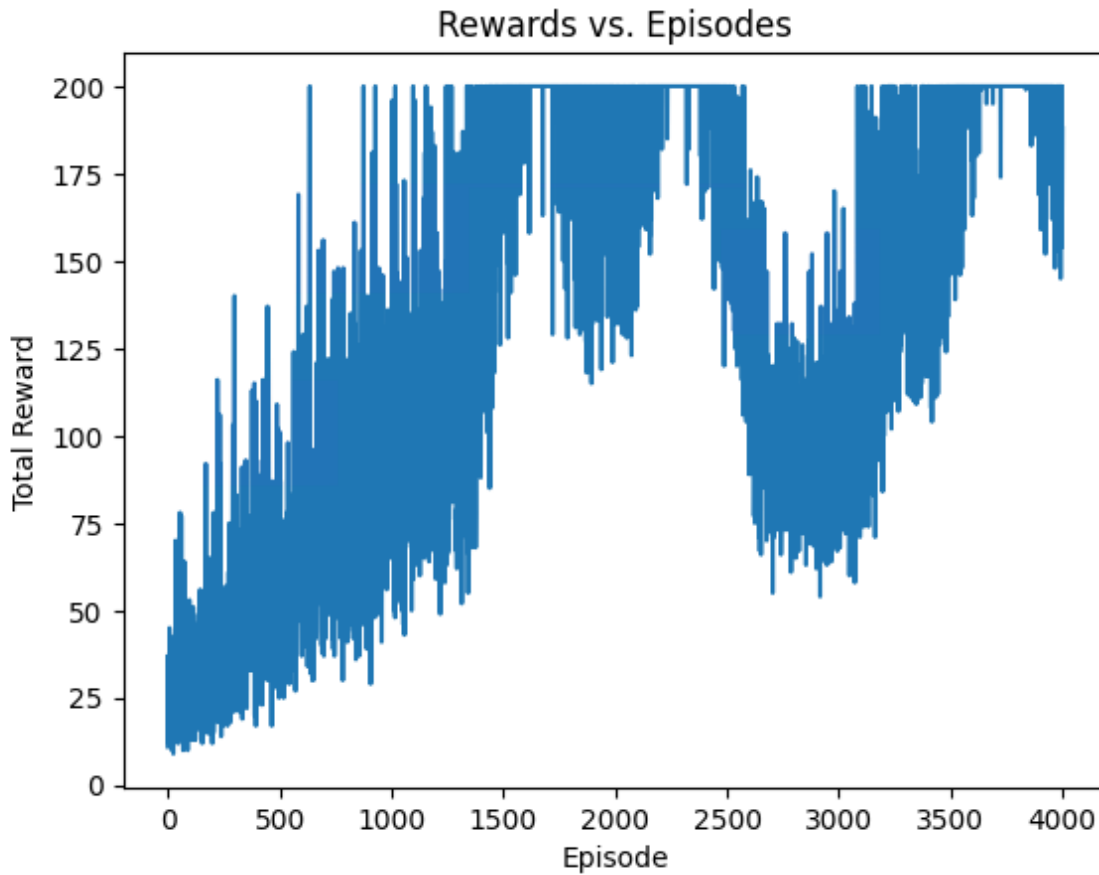
1. The discounted rewards were normalized
2. The probability vector of selected actions was saved in tensors instead of vectors for each episode, to ensure that information has been tracked within tensorflow and gradient tape.
3. The moving average of the 100 consecutive episodes was used in the plot

Some lines of code were changed briefly between Version1, and the Updated version, but the overall implementation is the same.

[Results](#)

[Problem1_Version 1.ipynb](#)

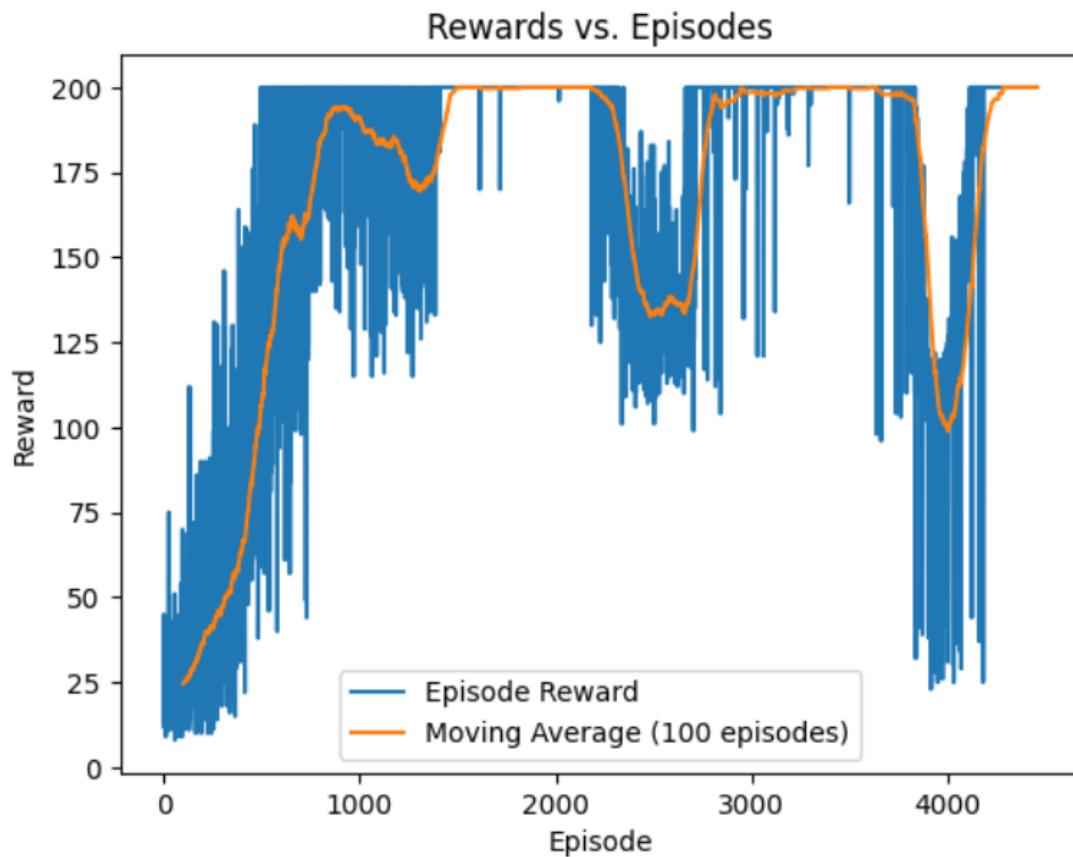
With version 1 of the code, and 4000 episodes, the model gives the following results.



The model clearly shows the learning of the agent, and although it fluctuates, it shows that it can obtain an average consecutive score of 100 episodes of more than 195. The suggestion to stop the model from fluctuation is either to use a decaying learning factor, or stopping the training at episode =2000 when the model converges.

Average Reward (Last 100 Episodes): 153.58
Average Reward (Last 100 Episodes): 156.6
Average Reward (Last 100 Episodes): 159.67
Average Reward (Last 100 Episodes): 164.92
Average Reward (Last 100 Episodes): 174.21
Average Reward (Last 100 Episodes): 180.55
Average Reward (Last 100 Episodes): 185.49
Average Reward (Last 100 Episodes): 190.04
Average Reward (Last 100 Episodes): 192.79
Average Reward (Last 100 Episodes): 195.05
Average Reward (Last 100 Episodes): 197.12
Average Reward (Last 100 Episodes): 197.75
Average Reward (Last 100 Episodes): 198.58
Average Reward (Last 100 Episodes): 199.6
Average Reward (Last 100 Episodes): 199.64
Average Reward (Last 100 Episodes): 199.64
Average Reward (Last 100 Episodes): 199.69
Average Reward (Last 100 Episodes): 199.74
Average Reward (Last 100 Episodes): 199.74
Average Reward (Last 100 Episodes): 199.99
Average Reward (Last 100 Episodes): 199.99
Average Reward (Last 100 Episodes): 199.82
Average Reward (Last 100 Episodes): 199.19
Average Reward (Last 100 Episodes): 198.23
Average Reward (Last 100 Episodes): 197.23
Average Reward (Last 100 Episodes): 195.59
Average Reward (Last 100 Episodes): 192.34
Average Reward (Last 100 Episodes): 190.41

[Problem1_Updated.ipynb](#)



Ideally, we should stop the training of the model earlier, once the average of the last 100 consecutive episodes converges to 200. This plot shows the noisiness of the model, and the variance in the policy learning process. The model keeps fluctuating if one proceeds in training it. The model also clearly stabilizes around 1700 episodes, however, neural networks are prone to catastrophic forgetting; with more training.

(note that the code `Problem1_Updated.ipynb`, was interrupted at ~ 4454 episodes, as the assigned number of batches was accidentally set to 6000 instead of 200, and there was no need to keep training the model further. However, the code runs correctly in deepdish and google colab with no issues).

Acknowledgments:

For debugging the code, understanding syntax and role of functions, and learning about different implementation methods, parameters:

- Tensorflow tutorials
- ChatGPT
- Discussions with classmates
- youtube